



INTRODUCTION TO ADVANCED ROBOTICS

PROJECT 5

SORTING GOOD PARTS

WHAT THIS PROJECT DECIDES

Project 4 asked a question about your hand. This one asks a question about an object, and it has to answer it the same way every time.

Put a part in the inspection zone and the program returns one of two words, plus a reason you can read from across the room:

```
PASS    Good Part (2841px)
FAIL    Too Small (940px)
FAIL    Shape Defect (Ratio: 1.62)
FAIL    No Item / Wrong Color
```

- *The reason is the whole point. A quality control system that says only "no" is useless, because somebody still has to work out what to do about it. Every `return` in this program carries its own explanation.*

THE ROBOT IS OPTIONAL THIS TIME

Read the robot setup in Project 4 again: the arm was created at the top level, outside any `try`, so no arm meant no program. This project does the opposite.

```
except Exception as e:  
    print(f"Robot hardware warning: {e}")  
    print("Continuing with OpenCV camera pipeline only...\n")
```

If the arm is missing, unplugged, or sulking, this program says so and carries on with the camera. You can do this entire project on a laptop.

Now notice something stranger. Look at what the arm is actually used for in this file: it is switched on and folded into a home position, and then never mentioned again. The inspection runs entirely on the camera. That is worth knowing before you go looking for the part the arm is supposed to be doing.

TWO QUESTIONS, ASKED IN ORDER

Everything in `inspect_part` is an answer to one of two questions, asked in this order and never any other:

- 1 Is anything there, and is it the right colour?**
Find the green pixels. If there are none, stop immediately.
- 2 Is it the right size and the right shape?**
Measure the biggest green shape and compare it to the spec.

Colour first, then size and shape. The order is not arbitrary — without the first question there is nothing for the second one to measure.

- *This is the shape of almost every inspection program you will ever write: cheap and decisive tests first, expensive and fiddly ones last.*

COLOUR IS THREE NUMBERS, BADLY

A pixel in the pictures you have been using is three numbers: blue, green, red, each from 0 to 255. To ask "is this green?" in that system you have to accept a fuzzy three-dimensional region — high green, low red, low-ish blue — and describe it as a shape in a cube. It works, and it is miserable to tune. Move the lamp and the region you need moves with it.

This is the same problem Project 4 had with colour order, seen from the other side. There, the fix was a swap that made detection quietly better. Here the fix is a whole colour space, and it is the single most useful idea in this project.

HSV PUTS COLOUR ON ONE AXIS

HSV is a different way of writing the same three numbers. Hue is the colour itself, as a single number around a wheel. Saturation is how strong the colour is. Value is how bright it is.

Because hue is one axis, "green" stops being a region and becomes an interval. Two numbers now describe it.

Saturation and Value are what make it survive bad lighting. A dark green and a bright green have the same hue, so a range on hue alone still finds both — which is exactly what you want when the lighting in the lab is not what you planned for.

You still have to give `inRange` a value for S and V, and this project gives them the widest legal range: 70 up to 255. Only the hue is being restricted. That is deliberate, and it is worth saying out loud in the debrief: this scanner accepts any brightness and any richness of green, and only cares which colour it is.

OPENCV STOPS AT 179

In mathematics, hue runs from 0 to 360. In OpenCV it runs from 0 to 179, because the value is squeezed into a single byte.

So green, which you might expect to sit at 120, is actually near 60. And the numbers in this file bracket it:

```
LOWER_GREEN = np.array([35, 70, 70])  
UPPER_GREEN = np.array([85, 255, 255])
```

35 to 85 is a band around 60 — green, with room on both sides. Hue 0 is red, and because the wheel closes, red is also at the far end of the scale. That wrap-around is why a colour at one end cannot be described by a single range.

A hue band this wide is a wide net. If the mask surprises you, print the hue of the pixels you did not want — read, then choose.

WHY THOSE BRACKETS ARE ARRAYS

A Python list would have been the obvious way to write those two lines. It is a NumPy array instead, because the next function is about to hand all three numbers to OpenCV at once and it wants something it can index numerically.

This is the first project where numpy earns its import. Project 4 imported it and never used it; here it appears twice — once for these bounds and once for the kernel in a few slides.

- *Every project in this module carries `import numpy as np` in the same place whether or not it needs it, so seeing it in the import block tells you nothing. What tells you something is where it gets used.*

INRANGE: YOUR FIRST THRESHOLD

One line, and it is the moment this project turns into computer vision:

```
hsv = cv2.cvtColor(roi_frame, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, LOWER_GREEN, UPPER_GREEN)
```

The mask is another picture, the same size as the region you passed in. Every pixel is now either 255 — in range, so it is green — or 0 — out of range, so it is black. Nothing in between.

This is the same idea as the thresholds in Project 4, but cruder and more useful: it does not produce a score, it produces an image you can look at. That is why this program opens a second window. When the mask does not match what you expected, you can see exactly which pixels are being counted and which are being thrown away.

THE MASK IS A PICTURE TOO

Two windows come up at the end of this program, and the second one is the reason this project is teachable:

```
cv2.imshow("Mission 02 - Project 05: QC Scanner", frame)
cv2.imshow("Mask View", mask)
```

The first window is what the part looks like to a person. The second is what the program actually believes: white where it thinks there is a part, black everywhere else.

- *When the two disagree, the mask is not lying and neither is the camera — your colour range is simply wrong. That single window turns an hour of guessing numbers into ten seconds of looking.*

SPECKLED MASKS LIE

A raw mask is never clean. Real images have sensor noise, and a single stray bright pixel inside the range becomes its own white dot. Left alone, those dots turn into their own contours later, and the "largest contour" picks whichever speck happens to be biggest.

The fix is two lines:

```
kernel = np.ones((5, 5), np.uint8)
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
```

`np.ones` builds a 5 by 5 block of ones — a small square of tool. `MORPH_OPEN` shrinks everything with it, then grows it back.

Shrinking then growing is an opening: it removes anything too thin to survive the squeeze and leaves the chunky shapes roughly as they were. It is the standard first move whenever a mask comes out speckled.

THE TRAP INSIDE OPENING

Here is the thing to be careful about, and it connects two constants that look unrelated.

A 5 by 5 opening deletes any blob smaller than about that size. So if a part is genuinely small — or far away, or only partly green — the noise filter removes it entirely, and by the time the rules run there is nothing left to measure.

- *A part that is too small is supposed to report `Too Small`. If the opening is aggressive enough, it reports `No Item / Wrong Color` instead — a completely different fault, pointing at the colour range rather than at the size. Same part, wrong diagnosis.*

This is the single most useful thing to understand about the whole project. When a fault message does not match the fault, suspect the step before the message, not the message itself. The mask window shows you this immediately: if the small part is not in the mask, it never reached the rules.

CONTOURS ARE OUTLINES

A mask is a field of loose white pixels. What the rules need is an object with a shape you can measure. That is what a contour is: the boundary traced around each connected white region.

```
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```

Two values come back. `contours` is the list you want. The second, thrown away here with `_`, is the hierarchy — the tree saying which contour is inside which, which this project never asks about.

- *`RETR_EXTERNAL` keeps only outer boundaries and ignores holes. `CHAIN_APPROX_SIMPLE` stores straight runs as single points instead of every pixel along them. That second flag is not a choice about correctness — it is a choice about how much memory the list uses, and it makes straight edges cheaper.*

FINDING NOTHING IS A RESULT

The next two lines were written for you, and they exist because the line after them would crash:

```
if not contours:  
    return "FAIL", "No Item / Wrong Color", mask
```

An empty list is falsy, so `if not contours` catches it. Without this, `max(contours, key=cv2.contourArea)` on the next slide would raise `ValueError: max() arg is an empty sequence` — and a quality control system that crashes on a bad part is worse than useless.

An empty zone is not an error condition here. It is a perfectly ordinary answer, and the program treats it as one. Notice also that all three returns from this function hand back `mask` as the third value, including this early one, so the caller always has something to display.

THE BIGGEST BLOB IS THE PART

The mask may contain several shapes. This program assumes the part is the largest one, and says so in the comment:

```
largest_contour = max(contours, key=cv2.contourArea)
area = cv2.contourArea(largest_contour)
```

Two different measurements of the same shape, and both are used. `max` searches the list using `cv2.contourArea` as its key, so it never builds a list of areas to compare. Then `cv2.contourArea` measures the winner, and that number goes straight into the size rules.

The assumption is the weak point, and it is worth being able to defend. Put a green pen on the table next to the part and the pen may well win, because it is a smooth solid shape while the part has holes and edges. Picking the largest contour is a decision, not a truth.

A BOX, NOT A ROTATED BOX

The next two lines measure shape, and the first one has a trap in its name:

```
x, y, w, h = cv2.boundingRect(largest_contour)
aspect_ratio = float(w) / h
```

`boundingRect` returns the smallest axis-aligned rectangle around the shape — aligned to the picture, never rotated. A part lying at an angle still gets an upright box. That is why the aspect ratio is a rough measure rather than a good one. A square rotated 20 degrees has a box that is wider than it is tall, and a quality check written this way will call it defective.

The `float()` is not decoration. In Python 3, `w / h` already gives a float, so the call is unnecessary here — but it is the habit you want from code written for Python 2, and it makes the intent obvious to whoever reads it next.

THREE RULES, FIRST ONE WINS

This is the only place in the project where a verdict is reached:

```
if area < MIN_AREA:
    return "FAIL", f"Too Small ({int(area)}px)", mask
if area > MAX_AREA:
    return "FAIL", f"Too Large ({int(area)}px)", mask
if abs(aspect_ratio - TARGET_ASPECT) > ASPECT_TOLERANCE:
    return "FAIL", f"Shape Defect (Ratio: {aspect_ratio:.2f})", mask
```

Three separate tests, each returning immediately. There is no score and no combined judgement — the first test that fails is the answer you get.

So a part that is both too small and the wrong shape reports only `Too Small`, and never mentions the shape. That is a deliberate trade: simple to read, simple to write, and it always tells you the first thing that is wrong. If your factory needed all the faults at once, this is the shape of code you would have to rewrite.

ROWS FIRST, THEN COLUMNS

This is the line students get wrong most often, and it is worth reading twice:

```
roi = frame[y1:y2, x1:x2]
```

A NumPy slice takes rows first and columns second. `y1:y2` picks the vertical span, `x1:x2` picks the horizontal span. It is the opposite order to how you say it out loud, and swapping them produces an error rather than a wrong answer — which is a small mercy.

The constants are in the same order, which helps and hurts in equal measure:

```
# Base ROI Coordinates [y_min, y_max, x_min, x_max] on 640x480 frame  
BASE_ROI = [120, 360, 200, 440]
```

Rows, then columns, in the comment, in the constant and in the slice. Consistent, and still the most common source of a transposed crop.

SCALE AND OFFSET AROUND THE MIDDLE

The inspection zone is not hard-coded. It is built at run time from a centre, a scale and an offset, so you can retune it without touching any arithmetic:

```
ROI_SCALE = 0.95      # 1.0 = original size, 0.5 = half size, 2.0 = double size
ROI_OFFSET_X = -15     # Horizontal offset (pixels), positive = right
ROI_OFFSET_Y = -70     # Vertical offset (pixels), positive = down
```

Scale 0.95 shrinks the box slightly. The offsets move it — negative X moves left, negative Y moves up. Three numbers, and you can retune the whole zone by editing three lines.

Note that 0.95 does not mean 5 percent smaller in each direction. The scale is applied to the width and height before the centre is recalculated, so it is genuinely a scale of the box and not a fudge.

CLAMPING, AGAIN

At the bottom of `calculate_roi_bounds` there are four lines that look like they are tidying up. They are not. They are the reason this function cannot crash.

```
new_x1 = max(0, min(new_x1, 640))  
new_x2 = max(0, min(new_x2, 640))
```

Push the offsets far enough and the box hangs off the edge of the picture. A NumPy slice with an out-of-range end is not an error — it silently returns a smaller region, or an empty one. These lines make sure the region is always inside the frame.

- *This is the same clamp as `map_value` in Project 4, doing the same job on a different type. Once you have seen it twice you start looking for it everywhere, and in this module it is never wrong.*

THE NUMBERS YOU WILL ACTUALLY TUNE

Four constants decide what passes, and all four are at the top of the file for exactly this reason:

```
MIN_AREA = 2000      # Minimum contour area (pixels)
MAX_AREA = 6000      # Maximum contour area (pixels)
TARGET_ASPECT = 1.0  # Expected width/height ratio (Square = 1.0)
ASPECT_TOLERANCE = 0.25
```

- `MIN_AREA` rejects specks and far-away parts
- `MAX_AREA` rejects parts that are far too big to be the same part
- `TARGET_ASPECT` 1.0 because the part is meant to be square
- `ASPECT_TOLERANCE` how far off square is still acceptable

They look like measurements of one specific part — and that is the problem. They describe the parts you had, not the parts you are supposed to make; derive them from the specification, not a sample.

STEP 1

Three imports, and for once every one of them is used:

```
import cv2
import numpy as np
import time
```

`cv2` for the camera, colour, contours and drawing. `numpy` for the colour bounds and the kernel. `time` for the three waits in the robot setup.

This is the smallest import block in the module so far. Project 3 needed `os` and `threading`, Project 4 needed `math` and `mediapipe`, and this one needs three. If an import is missing you will find out immediately with a `NameError` naming it, which makes this the easiest project in the module to get running.

STEP 2

The arm import, unchanged from every project before it:

```
from pymycobot.mycobot280 import MyCobot280
```

And this time it genuinely may go unused, because the robot setup is inside a `try` that carries on when it fails.

Import the name even if you are working without hardware. The import is harmless when there is no arm, and it means the one line you would have to change to enable the robot is already correct.

COPY IT, AND START IT ONCE

One file, and this time you do not need the arm at all:

```
scp M2-P5-Starter.py er@192.168.1.149:Documents
```

```
python M2-P5-Starter.py
```

Run it now, with every TODO still empty. It fails, but not where you would expect:

```
Robot ready.  
Traceback (most recent call last):  
  File "M2-P5-Starter.py", line 51, in <module>  
    LOWER_GREEN = np.array([35, 70, 70])  
NameError: name 'np' is not defined
```

Read those two lines together — the lesson. It prints `Robot ready.` and there is no robot: the empty TODO removed the connection code, so the `try` had nothing to fail on. Success means no exception.

STEP 3

Connect, wait, power on, wait. The two sleeps are not padding:

```
print("Connecting to myCobot280...")
mc = MyCobot280('/dev/ttyAMA0', 1000000)
time.sleep(0.5)
mc.power_on()
time.sleep(0.5)
```

The first wait is for the arm to finish booting after the serial port opens. The second is after `power_on()`, which is a command with real work to do behind it.

If any of this raises, the `except` catches it and the program continues. That means a genuinely broken arm is not a fatal error here — so if your camera window never appears, look for a message about the robot that you scrolled past without reading.

STEP 4

Fold the arm out of the way and leave it there:

```
home_pos = [0, 45, -90, -45, 0, 0]
mc.send_angles(home_pos, 50)
time.sleep(2.0)
```

- *Two things are new here. `send_angles` takes six joint angles, where Project 4 used `send_coords` and took millimetres — same robot, two completely different ways of telling it where to go. And the position is a list of six numbers, not a single coordinate.*

The two second wait afterwards is not a guess. It is roughly how long the arm takes to travel there, and the program does not do anything else until it has arrived.

Once this has run, the arm is not used again anywhere in Project 5. The inspection is entirely camera work. It is a slightly odd thing to write, and it is worth being able to explain why it is there: it puts the arm in a known safe position at the start of every run, which is a habit worth keeping in code that does move a robot later.

STEP 5

The whole ROI function, signature included. Sixteen lines, and most of them are arithmetic about the middle:

```
def calculate_roi_bounds():
    y1, y2, x1, x2 = BASE_ROI
    center_x = (x1 + x2) // 2
    center_y = (y1 + y2) // 2
    width = (x2 - x1) * ROI_SCALE
    height = (y2 - y1) * ROI_SCALE
    new_x1 = int(center_x - width // 2 + ROI_OFFSET_X)
    new_x2 = int(center_x + width // 2 + ROI_OFFSET_X)
    new_y1 = int(center_y - height // 2 + ROI_OFFSET_Y)
    new_y2 = int(center_y + height // 2 + ROI_OFFSET_Y)
    new_x1 = max(0, min(new_x1, 640))
    new_x2 = max(0, min(new_x2, 640))
    new_y1 = max(0, min(new_y1, 480))
    new_y2 = max(0, min(new_y2, 480))
    return [new_y1, new_y2, new_x1, new_x2]
```

Six steps hidden in there: unpack, find the centre, scale the size, apply the offset, clamp to the frame, return. The `//` is floor division, so a width of 239 becomes 119 rather than 120 — which is why the box can come out a pixel smaller than you asked for, and why nobody minds.

STEP 6

This one is already done for you. Look closely at what your file already contains:

```
def inspect_part(roi_frame):  
    """  
    Analyzes an item within the ROI frame.  
    Returns: status (str), reason (str), color_overlay (image)  
    """
```

The signature and the docstring are already in the starter file, so there is nothing to type for this TODO. The real work is the five steps inside it, and they are the next five slides.

- *Read that docstring carefully, because it tells you the contract: a status word, a reason, and an image.*
- *Every single `return` in this function must produce all three, which is why each one ends with `mask` even the ones that bail out early.*

STEP 7

One line: blue-green-red into hue-saturation-value.

```
hsv = cv2.cvtColor(roi_frame, cv2.COLOR_BGR2HSV)
```

It is the same colour conversion as Project 4, with a different answer. There the point was to give MediaPipe the order it expected; here the point is the space itself.

Notice this converts `roi_frame`, not `frame`. Only the part inside the inspection zone is ever converted, measured or judged. Everything outside it is ignored — which is what makes the zone adjustable at all.

STEP 8

The threshold, and the first moment this program actually decides anything:

```
mask = cv2.inRange(hsv, LOWER_GREEN, UPPER_GREEN)
```

One picture in, one picture out. Everything green becomes white, everything else becomes black, and the rest of the function works on the black and white version rather than on the original.

This is the line to change when the scanner accepts the wrong thing or rejects the right thing. Look at the mask window while you change it: you are adjusting which pixels count, and you can see exactly which ones you just gained or lost.

STEP 9

Clean the mask before trusting it:

```
kernel = np.ones((5, 5), np.uint8)
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
```

The kernel is 25 ones in a square. `np.uint8` is required — OpenCV will not accept a different type here, and it is the reason this line cannot simply be a Python list.

Keep an eye on the small parts while you work on this. If the mask loses something, the opening took it, and no amount of adjusting the colour range will bring it back. The Mask View window is where you catch that immediately.

STEP 10

Turn the white blobs into outlines:

```
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```

The mask is modified in place by this call, which is a long-standing OpenCV behaviour and worth knowing about: the `mask` you get back afterwards is not the mask you passed in.

That is not a problem here, because the only use of `mask` afterwards is to display it in the second window. But if you ever need the original mask back, take a copy before calling this.

STEP 11

Pick the part out of whatever else was in the zone:

```
largest_contour = max(contours, key=cv2.contourArea)  
area = cv2.contourArea(largest_contour)
```

This is the assumption of the whole project in two lines. It works beautifully on a clean background and fails quietly on a cluttered one.

If you want to see how much it depends on the assumption, put two parts in the zone at once. Only one is measured and the other is invisible to the program. The mask window will show you both, which is the quickest way to prove that "largest" is a policy rather than a measurement.

STEP 12

Measure the shape:

```
x, y, w, h = cv2.boundingRect(largest_contour)
aspect_ratio = float(w) / h
```

The `x`, `y` and `h` are measured and then not used. Only `w`, `h` and the ratio matter here — the rectangle is a local measurement, not the one drawn on screen. The box you see comes from the ROI, not from this.

That is a small thing worth noticing: two different rectangles are in play in this program. `boundingRect` describes the part, and it never leaves the function. The rectangle drawn on the picture later is the inspection zone, which is fixed. Keep them apart in your head or the debugging will confuse you.

STEP 13

The verdict, and the end of the function:

```
if area < MIN_AREA:
    return "FAIL", f"Too Small ({int(area)}px)", mask
if area > MAX_AREA:
    return "FAIL", f"Too Large ({int(area)}px)", mask
if abs(aspect_ratio - TARGET_ASPECT) > ASPECT_TOLERANCE:
    return "FAIL", f"Shape Defect (Ratio: {aspect_ratio:.2f})", mask

return "PASS", f"Good Part ({int(area)}px)", mask
```

The f-strings are doing real work here. Each message carries the number that caused the verdict, which is what turns "no" into "no, and here is by how much". Note the rounding on the ratio: two decimal places is enough to tell a 1.62 defect from a 1.05 pass, without a number too long to read from across a bench.

STEP 14

The camera, identical to Projects 3 and 4:

```
cap = cv2.VideoCapture(0)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
```

This time the resolution is not a suggestion. `BASE_ROI` is described in the comment as coordinates on a 640 by 480 frame, and the clamp in `calculate_roi_bounds` hard-codes 640 and 480 for the same reason.

If the camera gives you a different size, the ROI will be in the wrong place and the clamp will be wrong too, and neither will complain. This is the first project in the module where the frame size is load-bearing, so it is worth printing `frame.shape` once to confirm what you are actually getting.

STEP 15

Read the frame:

```
ret, frame = cap.read()
if not ret:
    break
```

Almost identical to Project 4, with one deliberate difference. There the line was `success, frame` and the body was `continue`; here it is `ret, frame` and the body is

`break`. Both are correct, and the difference is about what you want to happen. `continue` says this frame is bad, try the next one — right for a program where frames arrive at 30 a second and one bad frame means nothing. `break` says the camera is finished, stop the program — right when there is nothing to read and waiting will not help. A dropped frame and a closed camera look identical from inside the loop, and this project decides to treat both as the end.

STEP 16

Build the zone and cut the part out of the picture:

```
ROI_BOUNDS = calculate_roi_bounds()  
y1, y2, x1, x2 = ROI_BOUNDS  
roi = frame[y1:y2, x1:x2]
```

Three steps: get the bounds, unpack them, slice. The result is a NumPy array, not an OpenCV image object — which does not matter, because every function it is about to meet takes a NumPy array.

This runs inside the loop, every frame, even though the answer never changes unless you edit the constants. It is a function call per frame doing arithmetic on four numbers — utterly negligible, and it means you can move the zone without restarting the program.

STEP 17

One line, and it is the whole point of the project:

```
status, reason, mask = inspect_part(roi)
```

Three values back, unpacked into names that describe what they are. Compare that with Project 4, where the model returned nested lists you had to dig through — a function that hands back three well-named values is doing you a real favour.

Worth being precise about what the mask actually is. It is not a picture of the outlines — it is the cleaned mask, the one morphology produced and `findContours` read. Since OpenCV 3.2 that call does not modify the mask handed to it, so what comes back is exactly what went in: the same image the decisions were made from. That is why it is the one worth displaying.

STEP 18

The colour was already chosen for you, one line above this TODO:

```
status_color = (0, 255, 0) if status == "PASS" else (0, 0, 255)
```

```
cv2.rectangle(frame, (x1, y1), (x2, y2), status_color, 2)  
cv2.putText(frame, "INSPECTION ZONE", (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, status_color, 2)
```

The rectangle uses the ROI bounds from Step 16, not the bounding rectangle from Step 12. It is showing you where the scanner is looking, which is what you want while tuning the zone.

One colour value, decided once, used for the rectangle and the label. If you ever find yourself computing the colour in two places, that is the moment to notice you should have kept the line that was written for you.

STEP 18, CONTINUED

And the telemetry, anchored to the bottom of the picture:

```
h, w = frame.shape[:2]
cv2.putText(frame, f"STATUS: {status}", (20, h - 50), cv2.FONT_HERSHEY_SIMPLEX, 1.0, status_color, 3)
cv2.putText(frame, f"REASON: {reason}", (20, h - 15), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 2)
```

Those last two lines are the longest in the project, which is why they are set small here. In your file they are 112 and 108 characters, one line each, and you should type them exactly as they are.

Note `frame.shape[:2]` rather than the full shape — a shorter way of saying "height and width, ignore the channels"

Text at 1.0 scale with thickness 3 for the verdict, and 0.6 with thickness 2 for the reason. That is not decoration: the status is the answer and has to be readable from across the bench, while the reason is detail for whoever walks over to look. Two different sizes, two different jobs.

STEP 19

Two windows, and the last TODO:

```
cv2.imshow("Mission 02 - Project 05: QC Scanner", frame)
cv2.imshow("Mask View", mask)
```

The camera view for the human, the mask for you. Then the exit check, also already written:

```
if cv2.waitKey(1) & 0xFF == ord('q'):
    break
```

And the very last thing in the file:

```
finally:
    cap.release()
    cv2.destroyAllWindows()
```

No `mc.release_all_servos()` here — fine, the arm never moves, but the next project that moves the arm will want it back.

READING THE TWO WINDOWS

When something is wrong, look in this order:

- **The mask.** White where the program thinks there is a part. If the part is missing here, the problem is the colour range or the opening — nothing downstream has even seen it yet.
- **The inspection box.** Green means PASS, red means FAIL. This colour is the verdict, so it is the fastest read on the screen.
- **The reason line.** This is the fault, named in words with a number attached. Read it before changing anything.
- **The zone position.** If the part is half in and half out of the box, retune the offsets before you touch a single threshold.

The order runs from raw pixels to final verdict, which is the order the data flows in. Working backwards from the verdict is what turns a vague "it does not work" into a specific fault.

BEFORE YOU TUNE ANYTHING

What you see	What it means	What to do
No Item / Wrong Color, part present	Hue range wrong, or opening ate it	Watch the mask, then adjust the range
Too Small on a normal part	MIN_AREA too high, or it is too far away	Check the distance before the number
Too Large on a normal part	MAX_AREA too low	Compare against the real spec
Shape Defect on a square part	Part is rotated, or h is very small	Axis-aligned box cannot see rotation
Flashing between PASS and FAIL	Lighting is changing between frames	Fix the lighting, do not widen ranges
Reason never matches the fault	A step earlier removed the evidence	Read the mask window first
Robot warning in the terminal	No arm — and it does not matter here	Expected on a laptop, carry on

The last row is not a fault at all. Everything else on this table is, and each one points at a single step. "Reason never matches the fault" is the row worth memorising, because it is the one that saves the most time.

WHAT THE NUMBERS ACTUALLY COST

Five numbers decide everything, and every one of them is a trade rather than a setting to maximise:

- 1 35 to 85, the hue range**
wider accepts more shades of green, including olive and teal
- 2 5 by 5, the kernel**
bigger removes more noise, and more real parts with it
- 3 2000, the minimum area**
higher rejects more, including parts that are merely further away
- 4 6000, the maximum area**
lower starts rejecting parts that are closer than expected
- 5 0.25, the aspect tolerance**
tighter means squarer; too tight and a good part rotates into a fail

Same shape every time: too tight rejects good parts, too loose accepts bad ones. A decision, not a number to optimise.

CHALLENGE
YOUR MOVE

THE BRIEF

Build a scanner that judges a part and explains itself. Not "does it detect green" — that is a mechanism. The deliverable is a verdict, a reason, and a mask somebody can check your work with.

The arm is connected, powered and folded, and then left alone. That is deliberate, and you should be able to say why.



- ☐ Good parts report PASS with a plausible area.
- ☐ Bad parts fail with a reason that names the actual fault.
- ☐ The mask window shows the part, and only the part.
- ☐ The zone can be retuned by editing three constants.

YOUR MISSION: RIGHT FOR THE WRONG REASON COUNTS AS WRONG

Five things to get right before you touch a threshold:

- 1 A good part reports PASS**
Area between 2000 and 6000, ratio within 0.25 of 1.0
- 2 A part that is too small says Too Small**
Not No Item. If the opening ate it, fix the kernel
- 3 The wrong colour says No Item**
And the mask is empty, which proves it
- 4 Nothing outside the box is judged**
Move the zone with three constants, not arithmetic
- 5 Every reason carries a number**
The area in pixels, or the ratio to two places

Point 2 is the one that separates a scanner from a guessing machine. A verdict is only useful if it names the real fault, and the fastest way to name the real fault is to check that the evidence was still there when the verdict was reached.

YOUR DATA LOG

Measure before you adjust. Ten rows, and every cell is an observation.

What you put in the zone	Mask looks right?	Status	Reason	Area / ratio
Good part, well lit	_____	_____	_____	_____
Good part, dimmer	_____	_____	_____	_____
Part slightly too small	_____	_____	_____	_____
Clearly too small	_____	_____	_____	_____
Wrong colour	_____	_____	_____	_____
Square part, rotated	_____	_____	_____	_____
Nothing at all	_____	_____	_____	_____
Two parts at once	_____	_____	_____	_____
Hand steadying the part	_____	_____	_____	_____

- Rows 5, 6 and 7 are where the interesting faults live. If "clearly too small" and "slightly too small" both report *No Item*, the opening is removing them before the rules ever run, and no threshold you change afterwards will make them appear.

DEFINITION OF DONE

Tick every line. A scanner that opens two windows is not finished.

- ☐ All 19 TODOs filled in, with the comments copied as written.
- ☐ Ten data log rows completed from observation, not expectation.
- ☐ Every fault message reproduced at least once, deliberately.
- ☐ The mask window never shows anything outside the zone.
- ☐ All five tuning numbers explained in one sentence each.
- ☐ You can say why the arm is connected but never used.

The third line is the one people skip, and it is the one that proves the scanner works. A program that only ever reports PASS has not been tested. Deliberately feed it a blue part, a bolt, and your own hand.

GO FURTHER

Each of these is a small change to code you have already written.

- **Stop assuming the biggest blob.** Sort the contours by area and judge the second largest, or the one nearest the centre of the zone.
- **Inspect every part, not one.** Split the zone into a grid, run the same function on each cell, and list what you found.
- **Stop rotating into failure.** Replace `boundingRect` with `minAreaRect`, which returns a rotated box, and compare the four corner lengths instead of one ratio.
- **Inspect a second colour.** Copy the pipeline, change the two bounds, and prove that red needs two ranges because of the hue wrap-around.
- **Log the results.** Append each verdict to a file with a timestamp. Then you have data instead of opinions.

The third is the most valuable: an axis-aligned box cannot tell a square from a rotated square, and no amount of tolerance tuning fixes that.

MISSION

DEBRIEF